AR Object Gesture — Hand-Tracking Camera API

Overview

AR Object Gesture can use a local webcam stream for MediaPipe hand landmarks and a WebGL particle renderer for the object scene. Camera frames remain local to the browser. The visible camera monitor has been removed; the main particle stage remains pure black and uncluttered.

Camera lifecycle

The camera element is `video#cam` and is configured with `playsinline` and `muted`. The application owns camera startup through the `HandCamera` class:

```
const app = window.wonderSnap.app;
await app.startCamera(); // requests camera permission and starts trackin
app.stopCamera();        // stops the stream; no camera panel is rendered
```

The video source remains as hidden `video#cam` for optional hand tracking. There is no `.camera-monitor`, `#camStatus`, or camera toolbar panel in the UI.

Browser integration API

The app exposes a browser integration object at `window.wonderSnap`:

API	Description
`CATALOG`	Current object definitions, including `Atom` and `Vibranium`.
`CFG`	Runtime particle settings: `n`, `dpr`, and `trails`.
`app.startCamera()`	Starts camera permission flow and hand tracking.
`app.stopCamera()`	Stops the camera and tracking loop.
`app.cam.poll()`	Returns the latest detected hands when the camera is running.
`select(index, autoForm)`	Selects an object by catalog index.
`advance(seconds, hand, hand2)`	Deterministic integration hook for synthetic hand landmarks and browser tests.
`status()`	Returns current state, gesture, particle, camera, and selection telemetry.
`anchor(label)`	Returns screen coordinates for a labeled exploded part.
`pointAt(x, y, scale)`	Builds synthetic pointing-hand parameters for a screen location.

Status telemetry

`window.wonderSnap.status()` includes:

```
{
  name: 'Atom',
  state: 'formed',
  category: 'Elements',
  pose: 'pinch',
  pinch: true,
  pinchGap: 0.22,
  openness: 0.31,
  explode: 0.64,
  yaw: 0.42,
  pitch: -0.08,
  selected: 'Protons (+)',
  camera: { running: true, fps: 30, frames: 120 }
}
```

`pinchGap` is the normalized thumb-index distance. `0` means touching; approximately `0.30` is the maximum gap used by the debounced pinch recognizer.

Gesture mapping

General objects

- 🤘 Snap: summon or dissolve the particle cloud.
- Fist: form the selected object.
- Open hand: morph to the next wonder or explode machine parts.
- Twist / raise / lower: rotate and tilt the object.
- Point: dwell on a labeled part to open its explanation.
- Pinch: pull the selected part out or put it back.
- Two hands: zoom.

Atom and Vibranium

When an Elements object is formed:

1. Hold a pinch to enter atomic-lattice control mode.
2. Drag the pinched hand horizontally to rotate the lattice around the vertical axis.

3. Drag vertically to tilt the lattice.
4. Keep the pinch held and widen the thumb-index gap to increase explosion amount.
5. Bring the fingers together to reassemble the lattice.
6. Release the pinch to keep the current explosion amount and return to normal rotation/selection behavior.

The UI guide displays: `Pinch + drag — rotate lattice · widen pinch to explode`.

Interactive explanations

Atom exposes these labels and explanations:

- **Protons (+)** — positively charged particles in the nucleus.
- **Neutrons (0)** — neutral particles in the nucleus.
- **Nucleus** — dense central core containing protons and neutrons.
- **Electron shells** — quantized energy levels around the nucleus.
- **Electron cloud** — probability region where electrons are likely to be found.
- **Electrons (-)** — negatively charged particles occupying shells.
- **Valence electrons** — outermost electrons that control bonding and reactions.

Vibranium exposes:

- **Vibranium core** — fictional ultra-strong metal inspired by Iron Man 3.
- **Energy lattice** — glowing crystalline energy pattern.
- **Resonance veins** — stylized channels representing stored kinetic energy.

Selecting a part updates the part card, speaks the explanation when voice output is enabled, and adds the sentence to `status().spoken`.

Mobile defaults

For viewports up to 600px wide, the default particle count is 60,000. Larger mobile layouts default to 90,000. Mobile device pixel ratio is capped at 1.35 and particle trails are disabled by default. Query parameters override these defaults:

- `?n=90000` — explicit particle count.
- `?dpr=1` — explicit device pixel ratio.

- `?trails=1` — enable trails on mobile.

Testing example

```
const W = window.wonderSnap;
const atom = W.CATALOG.findIndex((d) ⇒ d.name === 'Atom');
W.select(atom);
await W.advance(0.14, W.synth({ cx: .33, cy: .58, s: .085, pose: 'snap_pr
await W.advance(0.20, W.synth({ cx: .33, cy: .58, s: .085, pose: 'snap_re
await W.advance(2.5, W.synth({ cx: .33, cy: .58, s: .085, pose: 'fist' }))
// Use a pinch gesture and vary the thumb-index gap to drive the lattice.
await W.advance(1.5, W.synth({ cx: .42, cy: .55, s: .085, pose: 'pinch' }))
console.log(W.status());
```